TalentDash

3-Day Engineering Trial Task

Frontend · Backend · Full-Stack · AI & Data Engineering

2025 | 72 Hours | Confidential — Do Not Distribute

72 Hours	**4 Roles**	**Ship It**	**Real Data**
Duration	Covered	Expectation	Standard

Please check these documents for a better understanding.

Talentdash_arch
TalentDash - Tech Stack
Competitive_Analysis_Labels_Bhargav.xlsx
Talentdash_Competitor_Audit and Masterlist of TalentDash_Bha…

What TalentDash Actually Is

Read this section before anything else. Most candidates who fail this trial do so because they built something technically fine but conceptually wrong.

TalentDash is a career intelligence platform. Not a job board. Not a company review site. Not a salary listing. The product is structured, comparable, decision-ready career data served at internet scale with near-zero infrastructure cost.

Here is the simplest way to understand it: imagine you are a software engineer in Bengaluru and you just got an offer from Amazon for ₹42 LPA. Before you decide, you want to know — is this a fair L4 offer at Amazon? What does Google pay for the same level? What do people say about Amazon's work culture? What were the interview rounds like? TalentDash answers all of that in one place. Every answer is sourced from structured, normalised data. No opinion columns. No star ratings without context. Just facts you can act on.

That sounds simple. The engineering problem underneath is not.

> *Core product principle: "Structured data → Comparable → Decision-ready." We are not building a content site. We are building a compensation intelligence engine that happens to look like a website.*

The Business Model Explained Simply

TalentDash makes money from traffic. Traffic comes from Google. Google sends traffic to pages that answer specific questions. So we build millions of pages that each answer one specific question:

- "What does a Software Engineer L4 earn at Amazon in Bengaluru?" → /salaries/software-engineer/amazon/bengaluru
- "What is it like to work at TCS?" → /companies/tcs/reviews
- "How hard is the Google PM interview?" → /interviews/google/product-manager
- "Which pays more — Flipkart or Meesho for an SDE-II?" → /compare/flipkart-vs-meesho

Each page costs us almost nothing to serve because it is prebuilt and stored on Cloudflare's global network. A page built once can be served a million times with zero server cost. This is why static-first architecture is not a technical preference — it is the business model.

> *The flywheel: more pages indexed by Google → more users → more salary submissions → better data → better pages → more pages. Every engineer's job is to make this flywheel spin faster and cheaper.*

Design System & Visual Identity

TalentDash's visual language is data-first, professional, and dense without feeling cluttered. The closest reference is how Airbnb communicates trust and clarity — every element earns its place, nothing is decorative. Unlike Airbnb's warm hospitality brand, TalentDash uses a colder, more analytical palette because the product is compensation intelligence, not lifestyle.

Study Airbnb's UI before starting the trial. Observe how they handle: search inputs with clear affordances, card density, trust signals, and mobile-first typography hierarchy. Apply that discipline — not their colors.

Color System

Element	Value	Usage
Primary Accent	**#FF5A5F — Coral Red**	**Primary CTAs, active states, salary highlights**
Deep Text	**#222222 — Airbnb Black**	**Main headings, company names, primary content**

Body Text	#484848 — Soft Dark Gray	Paragraphs, metadata, review content
Muted Text	#717171 — Neutral Gray	Secondary labels, timestamps, helper text
Surface Background	#FFFFFF — Pure White	Cards, tables, modals
App Background	#F7F7F7 — Soft Gray	Overall application background
Border Color	#EBEBEB — Light Border	Dividers, table borders, card outlines
Success Green	#008A05 — Positive Indicator	Salary increases, positive trends
Warning Orange	#FFB400 — Alert Indicator	Market caution indicators
Error Red	#D93025 — Risk/Error	Validation errors, rejection indicators
Hover Surface	#F2F2F2 — Hover Gray	Table row hover, interactive states

Typography

Element	Specification
Font Family	Inter (Primary), system-ui fallback
H1	36px, weight 700, line-height 1.1
H2	28px, weight 700
H3	22px, weight 600
Salary Figures	32–40px, weight 700
Body Text	16px, weight 400, line-height 1.6
Small Labels	13px, weight 500
Metadata	12px, weight 400

> *No component libraries (ShadCN, MUI, Chakra). Write Tailwind from scratch. The discipline of building your own components is what tells us whether you understand the UI, not just consume it.*

▸

Competitor Landscape — Know What Exists

Every engineer on TalentDash should have used all of these products. If you have not, spend 30 minutes before starting the trial.

Product	Their Strength	Where TalentDash Beats Them
levels.fyi	Gold standard for US tech salary data. Level-based, structured, trusted. The product we most closely reference.	Virtually no India data. No reviews, no interview experiences, no community. Limited to tech. We cover India deeply + multiple industries.
ambitionbox.com	Dominant in India for company reviews and salary data. High user trust among Indian professionals.	Data is unstructured and un-normalised. Salary ranges are too broad to be useful. No level system. UI/UX is outdated. No comparison engine.
glassdoor.co.in	Global brand recognition. Reviews + salaries in one place. Trusted by recruiters worldwide.	Paywalled. Increasingly ad-driven. Reviews are unverified. Salary data is noisy. Algorithm hides negative reviews.
teamblind.com	Best anonymous professional community. Real talk about companies, comp, and layoffs.	Forum-only. No structured salary data. No company pages. No SEO value. Entirely US-focused.
comparably.com	Strong culture and compensation analytics. Good UI for comparisons.	Low adoption in India. Limited data depth. Comparisons are surface-level.
LinkedIn Salarieslinkedin.com	Enormous user base. Salary insights built into job listings.	Paywalled. Vague salary bands with no level granularity. Not structured for comparison. No interview data.
payscale.com	Established global compensation database. Strong enterprise product.	Outdated consumer product. No community layer. No India-specific depth.

Platform Structure — Eight Product Areas

TalentDash has eight distinct product areas. They share a database but each has its own page types, SEO strategy, and data requirements. Know all eight. Your code will power all of them.

Area	What Users Do Here	Core SEO Pages Generated
Companies	Discover and research employers. View ratings, salary ranges, headcount, benefits,	talentdash.com/companies/{slug}talentdash.com/companies/{slug}/reviewstalentdash.com/companies/{slu

	funding stage, Glassdoor vs TalentDash score.	g}/salariestalentdash.com/companies/{slug}/interviews
Salaries	Look up compensation by role, level, company, location, industry. The primary product. Every page is a Levels.fyi equivalent for India.	talentdash.com/salariestalentdash.com/salaries/{role}talentdash.com/salaries/{role}/{location}talentdash.com/salaries/{company}/{role}talentdash.com/salaries/heatmap
Reviews	Read and submit anonymised employee reviews. Covers work-life balance, management quality, growth opportunities, culture fit.	talentdash.com/reviewstalentdash.com/reviews/{company}talentdash.com/reviews/{company}/{role}
Interviews	Find interview experiences by company and role. Understand difficulty, rounds, typical questions, offer rates.	talentdash.com/interviewstalentdash.com/interviews/{company}talentdash.com/interviews/{company}/{role}talentdash.com/profiles/{role}/interview-questions
Compare	Side-by-side comparison of companies or salary records. The "Dmart vs Amazon" or "offer letter A vs B" experience.	talentdash.com/compare/{company1}-vs-{company2}talentdash.com/compare (tool page)
Tools	Salary calculator, hike calculator, equity/ESOP calculator, offer comparison tool. High intent pages that attract decision-making traffic.	talentdash.com/tools/salary-calculatortalentdash.com/tools/hike-calculatortalentdash.com/tools/equity-calculatortalentdash.com/tools/offer-comparison
Community / Forum	Anonymous professional discussions. Company-specific boards, role-specific threads. TeamBlind-like layer.	talentdash.com/communitytalentdash.com/community/{company}talentdash.com/community/{topic}
Workplace Index	Composite scoring system that ranks companies on culture, compensation fairness, growth, D&I, WFH policy. Like a Michelin rating for employers.	talentdash.com/workplace-indextalentdash.com/workplace-index/{industry}talentdash.com/workplace-index/rankings

---

Tech Stack — Non-Negotiable

Every engineer works within this stack. No exceptions. These are not preferences — they are the decisions that keep infrastructure cost low and the product scalable.

Layer	Technology	Why This Specifically
Framework	Next.js 15 (App Router only)	React Server Components + generateStaticParams powers the entire static page engine. Pages Router is not used.

Styling	Tailwind CSS only	No ShadCN, MUI, or Radix. Writing utility classes directly builds discipline. Keeps bundle size minimal.
Rendering	SSG + ISR (Incremental Static Regeneration)	Static pages served from CDN edge. ISR refreshes stale pages without full rebuilds. SSR only for admin/auth.
Database	PostgreSQL via Neon (serverless)	Structured relational data. Acquisition-friendly. Excellent full-text search. Free tier handles MVP scale.
ORM	Prisma	Type-safe queries. Migration history in source control. Schema-as-code means any engineer can read the data model.
Hosting	Cloudflare Pages + CDN	Global edge delivery. Near-zero bandwidth cost. Ideal for SEO-heavy static sites. Predictable billing.
Storage	Cloudflare R2	Company logos, screenshots, sitemaps, JSON exports. Zero egress fees (unlike S3).
Job Queue	BullMQ + Upstash Redis	Background scraping, normalization, sitemap generation, AI enrichment jobs. Serverless-compatible.
Scraping	Python + Playwright / Scrapy	Playwright for JS-heavy sites. Scrapy for fast HTML crawling. Rate limiting required.
AI Layer	OpenAI + Claude + Gemini	Async batch jobs only — never on-request. Used for normalization, summaries, SEO content. Not real-time.
Search Phase 1	PostgreSQL Full-Text Search	tsvector on company + role fields. Free, no extra infra. Sufficient for MVP.
Search Phase 2	Typesense (when needed)	Typo tolerance and autocomplete when PostgreSQL search quality degrades at scale.
Auth	Clerk or Auth.js	For contributor logins, admin panel, review submission. Light-touch — not on public pages.
Analytics	PostHog (self-hosted or cloud)	Product analytics + SEO event tracking. Understand which pages drive conversions.

Rendering Decision Table — Memorise This

Page Type	Rendering Strategy	Reason
Homepage	ISR (revalidate: 3600)	Changes daily (trending companies, recent salaries). Too dynamic for full static build.
Company Page /companies/{slug}	Static (generateStaticParams)	Changes rarely. Pre-built for every known company at build time.
Salary Pages /salaries/{role}	Static	Core SEO asset. Must be fastest possible. Rebuilt when new data added via ISR trigger.

Location Salary Pages	Static	Millions of combinations — prebuilt for top 50 cities, ISR for the rest.
Comparison Pages	ISR (revalidate: 86400)	Combination pages cannot all be prebuilt. Generated on first request, cached after.
Search Results /search	Dynamic (RSC, no cache)	User-specific query. Cannot be prebuilt. Must still use RSC not client fetch.
Interview Pages	Static	Experience reports rarely change. Full static generation.
Review Pages	ISR (revalidate: 7200)	New reviews added frequently but not real-time critical.
Admin Panel	SSR (server-side rendered)	Auth required, personalised. Never cached.
Tools Pages	Static shell + client hydration	Tool UI is static. Calculations happen client-side in JS.

The Data Contract — Every Role Must Know This

This is the single most important reference in this document. Every role depends on this schema. The backend enforces it. The frontend renders it. The scraper produces it. The AI validates it. If any layer gets this wrong, every other layer breaks.

Field	Type & Constraint	Notes for Engineers
company	string, required	Normalised: always lowercase + trimmed. "Google", "GOOGLE", "google " → "google". Backend enforces this. Frontend displays the formatted version from a lookup table.
company_slug	string, required	URL-safe: "tata-consultancy-services". Auto-generated from normalised name. Powers /companies/{slug} routes.
role	string, required	Job title as entered. "Software Engineer", "Product Manager", "Data Analyst". Not normalised — preserved as submitted.
level_standardized	enum, required	L3 | L4 | L5 | L6 | SDE-I | SDE-II | SDE-III | Staff | Principal | IC4 | IC5. Free text like "Senior Engineer" is rejected at ingestion. The AI normaliser maps titles to this enum.
location	string, required	City name only. "Bengaluru", "Mumbai", "San Francisco", "Remote". No country suffix in the field. Country inferred from currency.
currency	enum, required	INR | USD | GBP | EUR. Determines display formatting and comparison normalisation.

experience_years	integer, required	Total years of experience. Reject if negative or > 50. If a range is given ("5–8 yrs") use the midpoint (6).
base_salary	number, required	Annual gross, in smallest currency unit (paise for INR, cents for USD). Reject if ≤ 0.
bonus	number, optional	Annual performance bonus. Defaults to 0 if not provided. Never null in the DB.
stock	number, optional	Annual RSU/ESOP vesting value. Defaults to 0. Never null in the DB.
total_compensation	number, COMPUTED	Always = base + bonus + stock. Never trust the client-submitted value. Recomputed on every ingest.
source	enum, required	CONTRIBUTOR | SCRAPED | AI_INFERRED. Affects confidence_score floor and display label.
confidence_score	float 0.0–1.0, required	Completeness + reliability score. CONTRIBUTOR with all fields = 0.9–1.0. SCRAPED with partial data = 0.4–0.7. Used to weight medians and filter low-quality records.
submitted_at	timestamp, auto	Server-side timestamp of record creation. Never trust client-submitted timestamps.
is_verified	boolean, default false	Set true after moderation or duplicate-check pass. Unverified records shown with a disclosure.

> *RULE: total_compensation is ALWAYS computed server-side. base + (bonus ?? 0) + (stock ?? 0). The ingest endpoint strips and recomputes this field regardless of what is submitted. Every engineer — backend, frontend, and AI — must know this.*

How the Data Pipeline Works — Read This Before You Build

This section explains where TalentDash data comes from, how it gets cleaned, and how it ends up on a page. If you are on the AI/Data team, this is your job. If you are on the frontend or backend team, you still need to understand this because your work depends on it.

The Pipeline: From Public Web to a Static Page

Stage	What Actually Happens
1 — Discover	The scraper (Python + Playwright) navigates to AmbitionBox, Glassdoor, or LinkedIn salary pages. It finds salary listings for a target role and company. Playwright handles JavaScript-rendered pages. Scrapy handles simpler HTML. Rate limiting is mandatory — never more than 1 request per 2 seconds per domain.

2 — Extract Raw	The scraper pulls the raw text around each salary record: company name as shown, role title as shown, salary text ("₹18–22 LPA"), location, experience description. This is messy. "Google India", "google pvt ltd", and "Google" are all the same company. "18–22 LPA" needs to become a number. "5+ years" needs to become an integer.
3 — LLM Normalisation	The raw extracted text is sent in batches of 10 to Claude or GPT-4o-mini with a structured prompt. The prompt instructs the LLM to output a JSON object matching the integration contract exactly. The model maps "Senior Software Engineer" → level: "L5", maps "18–22 LPA" → base_min: 1800000, base_max: 2200000, base_midpoint: 2000000, normalises the company name.
4 — Pydantic Validation	EVERY LLM response goes through a Pydantic model before it is used. The Pydantic model enforces the integration contract. If the LLM hallucinated a field, Pydantic catches it. If confidence_score is outside 0.0–1.0, it is rejected. Invalid records are logged with the rejection reason — never silently dropped.
5 — Deduplication Check	Before storing, check: does a record with the same company + role + level + location + base salary (within 10%) already exist and was submitted in the last 48 hours? If yes, skip it. This prevents the same Glassdoor page being scraped twice and polluting the data.
6 — Ingest via API	Valid records are POSTed to /api/ingest-salary. The backend recomputes total_compensation, sets source = 'SCRAPED', assigns a confidence_score floor of 0.5 for scraped records. Records with confidence < 0.4 are flagged for human review instead of auto-inserted.
7 — Static Page Regeneration	After a batch of new salary records is ingested, a background job triggers ISR revalidation on the affected company and role pages via Cloudflare's cache purge API. The next request to that page rebuilds it with fresh data. Users see updated pages without a full rebuild.

> *The scraper will get blocked. AmbitionBox returns CAPTCHAs after 50 requests. Glassdoor detects headless browsers. Your job is to handle this gracefully — add jitter to request timing, rotate user-agents, implement retry logic with backoff, and document every anti-bot measure you encountered and how you handled it.*

FRONTEND ENGINEER — 3-Day Trial Task

Frontend — What You Are Building and Why

The most visited page on TalentDash will be a salary table. Not a homepage. Not a blog post. A dense, filterable, level-aware compensation table that answers: "What does a Software Engineer at this company earn at this level in this city?"

Your job is to build that experience so well that a first-time user can find and compare a salary in under 10 seconds without any onboarding. The page must be fast enough to rank on Google (LCP <

2 seconds), structured enough for Google to read the data (JSON-LD schema markup), and correct enough that a candidate would trust it when negotiating an offer.

You are building against a mock JSON seed file. No database, no backend. Your task is pure frontend. But the architecture must be correct — React Server Components, static generation, proper component separation — because this code will ship to production.

F1 — Project Setup

What to deliver

- Next.js 15 with App Router, TypeScript strict mode, Tailwind CSS
- Folder structure: app/ (routes and layouts), components/ui/ (primitive components), components/features/ (product components), lib/ (utilities), types/ (TypeScript interfaces)
- No ShadCN, no MUI, no Radix, no Headless UI. Custom Tailwind components only.
- Path aliases: @/components, @/lib, @/types — configured in tsconfig.json
- A mock JSON seed file at lib/mock-data.ts — 50+ salary records matching the integration contract shape exactly
- ESLint + Prettier configured. Code that does not lint does not commit.

F2 — Salary Table Page /salaries

What to deliver

- Static page using React Server Components. Reads from the mock data file at build time.
- Table columns (in order): Company · Role · Level · Location · Experience · Base Salary · Stock · Total Comp
- Level column renders as a badge — different background color per tier: L3/SDE-I = slate, L4/SDE-II = blue, L5/SDE-III = indigo, L6/Staff = purple, Principal = navy
- Total Comp is the dominant number on each row — larger font, bold, data blue (#0369A1)
- Missing bonus/stock renders as "—" (em dash). Never undefined, never NaN, never blank.
- Filter bar: Company text search (debounced, 300ms), Role dropdown, Level multi-select checkboxes, Location dropdown, Currency toggle (INR / USD)
- Filters are URL-encoded: /salaries?company=amazon&level=L4&location=bengaluru — shareable link that pre-fills filters on load
- Sort by Total Comp descending by default. Click column headers to sort ascending/descending.
- Pagination: 25 rows per page. Previous / Next controls. Shows "Showing 26–50 of 312 records"
- Empty state when filters return 0 results: "No records found for these filters. Try removing a filter." with a clear-all link

The bar

- **Filters must actually work. We will test every combination. Filter + sort + pagination must work together correctly.**
- **Currency toggle must reformat all salary figures. ₹42,00,000 → $50,400 using a consistent conversion rate stored in a config file.**

F3 — Company Page /companies/[slug]

What to deliver

- Static page. generateStaticParams() pre-generates one page per unique company in the mock data.
- Company header: name (formatted, not lowercase slug), industry tag, founding year, headcount range, headquarters location
- Compensation overview: Median Total Comp badge (computed from all records for this company), min–max TC range, number of records
- Level distribution bar: horizontal stacked bar showing what percentage of records are L3, L4, L5, L6, etc. for this company
- Salary list: a filtered version of the salary table showing only this company's records. Same columns, same badge system.
- "Compare" button that links to /compare?c1={slug} pre-filling the first comparison slot

The bar

- **The median TC must be computed from the data, not hardcoded. If we add 10 new records for a company in the seed file, the median badge must update automatically.**
- **The level distribution bar must render correctly even if a company only has 1 level in the data.**

F4 — Compare Page /compare

What to deliver

- Client component (this is one of the few justified uses of 'use client' in the codebase)
- Two dropdown selectors: select any two salary records from the seed data
- Side-by-side breakdown: Company, Role, Level, Location, Experience, Base, Bonus, Stock, Total Comp
- Delta column: for each numeric field, show the difference. +₹4,00,000 in green, -₹2,00,000 in red
- Winner badge: the record with higher Total Comp gets a "Higher TC" chip in data blue
- URL state: selecting record A and B updates the URL to /compare?s1={id}&s2={id}. Sharing this URL loads the same comparison.

The bar

- **The delta calculation must be correct. Total Comp delta = TC of record A minus TC of record B. Positive = record A pays more. Negative = record B pays more.**

F5 — SEO Requirements (Non-Negotiable)

A page without correct SEO metadata is invisible to Google. Every page you build must have:

- Title tag: "Software Engineer Salaries at Amazon India — L3 to L5 | TalentDash"
- Meta description: accurate, specific, includes key terms naturally
- Canonical URL: prevents duplicate content penalties
- JSON-LD structured data: schema.org/JobPosting or schema.org/Dataset on salary pages. Google uses this to show rich results.
- Open Graph tags: og:title, og:description, og:url for social sharing

- Correct H1 on every page — matches the primary search intent

> *SEO is not a bonus task. It is a core deliverable. A salary table page with no JSON-LD structured data is a salary table page that will never rank on Google. TalentDash's entire acquisition strategy depends on organic search.*

F6 — Performance Requirements

- LCP (Largest Contentful Paint) must be under 2 seconds on a simulated 4G mobile connection
- No layout shift: skeleton loaders or reserved space for all async UI. CLS < 0.1.
- JS bundle: keep client-side JavaScript minimal. The salary table is a server component — it must ship zero client JS by default.
- Images: company logos use next/image with explicit width/height. No layout shift from images.

F7 — Edge Cases That Will Be Tested

- All filter combinations: company + role + level + location all applied simultaneously
- Empty results state from filters
- Single record for a company (level distribution bar with 100% of one level)
- Record with no bonus and no stock (shows "—" in both, TC = base salary exactly)
- Very long company name (40+ characters) — must not break table layout
- Very large salary number (₹4,00,00,000) — must format correctly with Indian lakh/crore system
- Company page accessed directly via URL with a valid slug vs an invalid slug (404 handling)

BACKEND ENGINEER — 3-Day Trial Task

Backend — What You Are Building and Why

TalentDash's data quality is the product. If "google" and "Google Inc." are stored as two different companies, every aggregate calculation is wrong. Every median. Every level distribution. Every comparison. The backend is where data integrity is enforced — not best-effort, not eventually consistent, but hard-rejected at the boundary.

You are building the ingestion pipeline and the query API. Every frontend page and every AI enrichment job will depend on what you build here. The schema decisions you make in Prisma will be the schema that powers millions of page views. Take the design seriously.

B1 — PostgreSQL Schema via Prisma

What to deliver

- schema.prisma with Salary model, Company model, and all fields from the integration contract
- Company is a separate model with a one-to-many relationship to Salary — not just a string field on Salary
- Level is a Prisma enum — not a String. Invalid levels are rejected at the database constraint level, not just in application code.
- Migration files in prisma/migrations/ — not just schema.prisma. Migration history must be in source control.
- Indexes: composite index on (company_id, level, location) for the primary filter query. Index on total_compensation for sort. Index on submitted_at for recency sort.

Full Prisma Schema Specification

Model / Field	Specification
Company model	id (UUID), name (display, "Google India"), slug (unique, "google"), normalized_name (lowercase indexed, "google"), industry (string), headquarters (string), founded_year (int, nullable), headcount_range (string, nullable), created_at, updated_at
Salary model	id (UUID), company_id (FK → Company), role (string), level (enum: L3|L4|L5|L6|SDE_I|SDE_II|SDE_III|STAFF|PRINCIPAL|IC4|IC5), location (string), currency (enum: INR|USD|GBP|EUR), experience_years (int), base_salary (bigint), bonus (bigint, default 0), stock (bigint, default 0), total_compensation (bigint, COMPUTED), source (enum: CONTRIBUTOR|SCRAPED|AI_INFERRED), confidence_score (Decimal, 0.0–1.0), is_verified (bool, default false), submitted_at (DateTime)
Indexes	@@index([company_id, level, location]) — primary filter path@@index([total_compensation]) — sort path@@index([submitted_at]) — recency path@@index([location, level]) — geo-level filter path
Constraints	total_compensation MUST be computed in the application layer, not trusted from input. confidence_score CHECK constraint 0.0–1.0. experience_years CHECK > 0 AND < 51.

B2 — POST /api/ingest-salary

What to deliver

- Accepts the integration contract JSON body
- Validation pipeline (in order): check required fields present → check types correct → check level is valid enum value → check experience_years > 0 and < 51 → check base_salary > 0 → check confidence_score between 0.0 and 1.0
- Normalisation pipeline: company name → lowercase + trim + strip punctuation. Find or create Company record by normalized_name.

- Recompute total_compensation = base_salary + (bonus ?? 0) + (stock ?? 0). Strip any client-submitted total_compensation.
- Duplicate check: if a record with same company_id + role + level + location has been submitted in the last 48 hours with base_salary within 10% of this record, return 409 Conflict with a clear message.
- Return 400 for validation failures with { error: true, field: 'level', message: 'Level must be one of: L3, L4, L5...' }. Per-field errors, not a single generic error string.
- Return 201 Created with the full stored record including computed total_compensation on success.

B3 — GET /api/salaries

What to deliver

- Query parameters: company (string, partial match), role (string, partial match), level (enum, exact), location (string, partial match), currency (enum), sort (total_comp_desc|total_comp_asc|date_desc), page (int, default 1), limit (int, default 25, max 100)
- Pagination: page-based. Response includes { data: [...], meta: { total, page, limit, totalPages } }
- Returning rows unbounded (no pagination) is a hard failure. We will test with a 10,000 row seed.
- company and role filters use PostgreSQL ILIKE for case-insensitive partial matching — not exact string equality
- Response time for a filtered query on 10,000 rows must be under 200ms. If the query is slow, add indexes.

B4 — GET /api/companies/:slug

What to deliver

- Returns: company metadata, full salary list for this company, median_total_compensation (computed server-side), level_distribution object
- median_total_compensation: sort all total_compensation values for this company, return the middle value. Do NOT return the average — return the true statistical median.
- level_distribution: { L3: 12, L4: 34, L5: 19, SDE_I: 8, ... } — counts per level for this company
- Returns 404 with { error: true, message: 'Company not found' } for unknown slugs
- Salary list in the response must be sorted by total_compensation descending

B5 — GET /api/compare

What to deliver

- Query parameters: s1 (salary record UUID), s2 (salary record UUID)
- Returns both full records plus a delta object: { base_delta: number, bonus_delta: number, stock_delta: number, tc_delta: number, experience_delta: number }
- Delta = record_1_value minus record_2_value. Positive = record 1 is higher. Negative = record 2 is higher.

- Returns 404 if either ID is not found. Returns 400 if IDs are identical.

B6 — Seed Script

What to deliver

- prisma/seed.ts with 60+ realistic records across: Google, Amazon, Meta, Microsoft, Flipkart, Meesho, NVIDIA, TCS, Infosys, Wipro, Razorpay, Zepto
- Records must span all levels (L3 through Principal), multiple cities (Bengaluru, Mumbai, Hyderabad, Pune, Delhi, San Francisco, London), both INR and USD
- Include intentional edge cases in the seed: one record with zero bonus, one with zero stock, one with very high equity, one with Principal level
- Normalisation must be demonstrated in seed: include company names like "Google India", "GOOGLE", "google" — all must resolve to the same Company slug "google"

B7 — Edge Cases That Will Be Tested

- POST /ingest-salary with negative base_salary → must return 400, not store the record
- POST /ingest-salary with level = "Senior Software Engineer" (not in enum) → must return 400
- POST /ingest-salary with total_compensation pre-filled incorrectly → must recompute and store the correct value
- GET /salaries with all filters applied simultaneously
- GET /companies/nonexistent-slug → 404
- GET /compare?s1=valid_id&s2=same_id → 400
- GET /salaries?limit=10000 → must cap at 100 and return meta.limit = 100

FULL-STACK ENGINEER — 3-Day Trial Task

Full-Stack — What You Are Building and Why

Full-stack on TalentDash means you own a feature from the PostgreSQL table definition to the rendered HTML that a user sees. There is no handoff. You design the schema, build the API, wire the frontend to it, deploy it, and verify it works with real data.

This is the hardest role in the trial. You are expected to deliver everything in the Backend section and everything in the Frontend section, connected together. The difference from doing both separately is that you must make the integration decisions — when does the frontend call the API? When does it use static generation? How does ISR revalidation trigger? These are judgment calls that only a full-stack engineer can make.

FS1 — Everything from Backend B1 through B7

Build the complete backend: Prisma schema, migrations, seed script, all four API endpoints, validation, normalisation, duplicate detection. All backend requirements apply in full.

FS2 — Everything from Frontend F1 through F7

Build the complete frontend: salary table, company page, compare page, SEO markup, performance requirements, all edge cases. All frontend requirements apply in full.

FS3 — The Integration Layer (Full-Stack Only)

generateStaticParams from real database

- The company page /companies/[slug] must use generateStaticParams() that queries the Neon database at build time to get all real company slugs
- Not a hardcoded array of slugs. Not a static JSON file. A live database query at build time.
- If a new company is added to the database, the next deployment automatically adds that company page

End-to-end data flow

- The reviewer will run: npx prisma db seed, then visit /companies/amazon, and expect to see real data from the database
- The reviewer will call POST /api/ingest-salary with a new record, then visit /salaries, and expect to see the new record after an ISR revalidation cycle
- Data must flow: database → API route → React Server Component → HTML. No mocked arrays in components.

Cache headers

- GET /api/salaries response must include Cache-Control: s-maxage=300, stale-while-revalidate=3600 so Cloudflare CDN caches it
- GET /api/companies/:slug response: Cache-Control: s-maxage=3600, stale-while-revalidate=86400
- Document in your README why you chose these TTLs.

FS4 — Trade-Off Decisions (Document These)

Your README must include a section called "Architecture Decisions" that covers:

- Why you chose static vs ISR vs dynamic for each page type
- Why your pagination is page-based rather than cursor-based (or vice versa)
- What you would build differently with another day
- What you did NOT build and why (scope choices under time pressure)

> *A full-stack candidate who ships a disconnected frontend (data mocked, not from the real API) has not completed the task. The end-to-end connection is the whole point of the role.*

AI & DATA ENGINEERING — 3-Day Trial Task

AI & Data Engineering — What You Are Building and Why

TalentDash's data moat is the only thing that separates it from a static website. The quality of that data — how clean it is, how normalised, how deduplicated — determines whether a user trusts the platform enough to make a career decision based on it.

Your job is to build the pipeline that creates that moat. You are building the system that continuously ingests raw salary data from the public web, sends it through an LLM to normalise it, validates every field strictly, deduplicates against existing records, and stores clean data that the backend team can serve.

Nobody on this team wants to manually clean salary data. That is your pipeline's job.

A1 — Web Scraper (Python + Playwright)

What to deliver

- Python script using Playwright (headless Chromium) targeting AmbitionBox salary listings
- Target pages: ambitionbox.com/salaries for Software Engineer and Data Analyst roles
- The scraper must handle pagination — not just scrape page 1 and stop
- Rate limiting: random sleep between 1.5 and 4 seconds between requests. Fixed-interval scraping gets you blocked.
- User-agent rotation: use at least 3 different realistic user-agent strings
- Per-record try/except: if one record fails to parse, log the error and continue to the next. Do not crash the entire run on one bad record.
- Output: list of raw extracted dicts with keys: raw_company, raw_role, raw_salary_text, raw_location, raw_experience

The bar

- **The scraper must produce at least 60 raw records across at least 6 different companies before LLM processing.**
- **If the target site blocks you: document the block in your README, explain how you detected it, and describe your solution. This is evaluated.**

A2 — LLM Normalisation Layer

What to deliver

- Async batch processor: groups raw records into batches of 10 and sends each batch to the LLM in a single API call
- Use Claude claude-haiku-3-5 or GPT-4o-mini — not expensive models. Cost optimisation is part of the evaluation.

- The prompt must instruct the LLM to return a JSON array where each element matches the integration contract exactly
- The prompt must explicitly handle edge cases: salary ranges (return the midpoint), ambiguous levels (return most likely enum value with lower confidence_score), missing fields (return null for optional fields, not a guess)
- Error handling per record: if the LLM returns a valid array but one element is malformed, log that element and process the rest. Do not fail the entire batch.
- Log the full prompt and response for the first batch in your README as a sample — we need to see your prompt design

A3 — Pydantic Validation Layer

What to deliver

- A Pydantic model SalaryRecord that enforces the integration contract
- Field validators: company must be at least 2 chars after normalisation, experience_years must be positive and < 51, base_salary must be > 0, confidence_score must be 0.0–1.0, level must be in the allowed enum
- A validate_record(raw_dict) function: returns (SalaryRecord, None) on success or (None, ValidationError) on failure
- Rejection log: every rejected record is written to a rejections.jsonl file with the raw input and the validation error. We will read this file.
- Never store a record that fails Pydantic validation. Never. The LLM is not reliable — Pydantic is the final authority.

A4 — Company Name Normalisation

What to deliver

This is the hardest part of the data pipeline. The same company appears in hundreds of ways across the public web. Your normalisation function must handle all of these:

Raw Input	Normalised Output
"Google India Pvt. Ltd."	"google"
"GOOGLE"	"google"
"Google " (trailing space)	"google"
"Tata Consultancy Services"	"tcs"
"TCS Ltd."	"tcs"
"Tata Consultancy"	"tcs" (requires alias table)
"amazon.com"	"amazon"
"Amazon Web Services"	"aws" or "amazon" (document your choice)
"Infosys BPO"	"infosys" (document your choice)

"Wipro Technologies"	"wipro"
"Flipkart Internet Pvt Ltd"	"flipkart"

- Build a normalisation function with two layers: (1) programmatic rules — lowercase, trim, strip legal suffixes ("pvt ltd", "inc", "ltd", "llc", ".com"), (2) alias lookup table for known variants ("tata consultancy" → "tcs")
- The alias table must be a separate data file (aliases.json or aliases.py) — not hardcoded inside the normalisation function
- Document in your README: show at least 5 normalisation examples with the input → output

A5 — Level Mapping System

What to deliver

Raw job titles and experience descriptions must map to the TalentDash level enum. Build a two-layer mapping system:

- Layer 1 — Rule-based: a mapping dict that handles known exact titles. "Software Engineer III" → L5, "Senior Software Engineer" → L4/L5 (ambiguous), "Staff Engineer" → Staff, "SDE-II" → SDE_II
- Layer 2 — LLM fallback: if rule-based returns ambiguous or no match, include the title and experience_years in the LLM normalisation prompt and ask the model to classify
- Confidence scoring: rule-based match = confidence 0.85+. LLM classification = confidence 0.6–0.8. No match = confidence 0.4 and flag for review.

A6 — Storage & Quality Report

What to deliver

- Store validated records: either POST to /api/ingest-salary if the backend is running, or directly write to Neon PostgreSQL using psycopg2 or asyncpg
- A quality report printed at the end of the pipeline run:
  - Total records scraped: X
  - Records passed LLM normalisation: X
  - Records passed Pydantic validation: X
  - Records rejected (with breakdown by reason): X
  - Records stored successfully: X
  - Null rate per field: documented per field
- README must show a side-by-side sample: raw scraped text alongside the final normalised JSON record that was stored

A7 — Deduplication Job

What to deliver

- Before storing a record: query existing records for the same company_id + role + level + location
- If a similar record exists (base_salary within 10%) and was submitted within 48 hours, skip and log as duplicate
- A deduplicate_existing_records() function that can run on the full database: finds records with identical company + role + level + location + base within 5% and flags all but the most recent one with is_verified = false

---

Communication Protocol — Non-Negotiable

This is not bureaucracy. This is how good engineers communicate. Being blocked silently for 4 hours is a red flag. Asking a smart question after 2 hours of trying is a green one.

	Short update message. Format: "Built [X]. Working on [Y] next. Blocked on [Z] — trying [approach]." Not a paragraph. Three lines maximum.
	Midpoint check. Share a working GitHub link or a live URL. If you are behind, say so and say what you are cutting and why.
	Final submission. Live URL + GitHub repo. A README that actually works. A short message (5 sentences) on the hardest decision you made and why.
Anytime — Blocked	If you have been stuck on the same problem for more than 2 hours, message the team with: what you are trying to do, what you have tried, what you think the issue is. Do not go silent.
Tone	Factual and short. "ISR cache not revalidating after POST. Tried revalidatePath('/salaries') — no effect. Checking Cloudflare cache headers." This is the right format.

Submission

GitHub Repo	Clean commit history — not one giant commit on Day 3. README with: architecture overview, how to run locally in under 5 minutes, all env vars documented with examples, live URL, one paragraph on trade-off decisions made.
Live URL	Deployed to Vercel or Cloudflare Pages (frontend/full-stack) or Railway/Render/Fly.io (backend/AI). Must be publicly accessible. "Works on my machine" is a hard failure for all roles except AI pipeline (where local execution is acceptable if deployment is not feasible in 72 hours).
README Quality	The README is evaluated. We will try to run your project using only the README. If it takes more than 5 minutes or fails, that counts against you. Include: env vars, database setup, seed command, start command, live URL.

AI Role — Data Sample	Show in the README: 3 raw scraped records side-by-side with their normalised output. Show 2 records that were rejected and the rejection reason. Show the quality report output from a real pipeline run.

How We Evaluate — In Order of Weight

#	Criterion	What We Actually Check
1	It works and is deployed	We open the live URL. Does the salary table load? Do filters work? Is real data showing? Does the API return correct shapes? If the core path fails on first attempt, review stops.
2	Data integrity (Backend & FS)	We POST a salary with negative base_salary, an invalid level, a duplicate record. Does the API reject correctly? Is total_compensation always computed, never trusted?
3	Static architecture	We run Lighthouse on the salary table page. Is LCP < 2s? Does the page use RSC correctly? Are generateStaticParams used? Is there unnecessary SSR?
4	Data pipeline quality (AI/Data)	We read the rejection log and the quality report. Is the normalization function handling the TCS/Tata edge case? Is Pydantic rejecting bad LLM output?
5	Code organisation	We read the source. Is there one 600-line page.tsx, or a clean component hierarchy? Is lib/db.ts separate from API handlers? Are Prisma queries in the right layer?
6	Edge case handling	We test every edge case listed in each role section. Salary table with 0 results. Company page with 1 record. API with all filters combined.
7	Communication	Did you send the Day 1 update? Were blockers flagged early? Did your README explain the decisions you made?

What will not pass

- Frontend: salary table with no working filter functionality — just a static list
- Frontend: using ShadCN or MUI when the task explicitly says not to
- Frontend: no live URL on Day 3
- Backend: total_compensation stored as whatever the client sends — no recomputation
- Backend: GET /salaries returning all rows with no pagination
- Backend: "google" and "Google" treated as different companies
- Full-Stack: frontend data is mocked — not connected to the real API/database
- AI/Data: LLM output stored directly without Pydantic validation
- AI/Data: level field stored as free text — "Senior Software Engineer" in the database
- All roles: no README, or README that does not produce a running project

Rules

AI tools	Use them. Claude, GPT-4, Cursor, Copilot. We use them too. The evaluation is on the system you build and the decisions you can explain — not whether you typed every line manually.
Tech stack	Stick to it. Next.js 15 App Router, Tailwind, Prisma, Neon. No Pages Router. No component libraries. No Kubernetes. If you think there is a good reason to deviate, message us first.
Scope cuts	72 hours is not enough to build everything perfectly. Cut deliberately. A salary table with solid filters and correct SEO beats a half-finished compare page and a broken company page. Document what you cut and why.
No auth	There is no authentication in this trial. No login walls, no session handling, no JWT. The scraper and admin endpoints are open. This is intentional — keep the scope clean.
Deadline	72 hours from when this document is shared. No extensions unless agreed before the deadline. Partial work submitted on time is evaluated. Missing the deadline entirely is not.
Questions	If something is genuinely ambiguous, ask once. Do not ask whether your approach is correct — that is the task. Do not wait 6 hours before asking.